

Linear_algebra

April 14, 2026

1 Linear Algebra

1.1 Contents

1. Linear Algebra

- Vectors
 - Intro
 - Set of all vectors
 - Physical quantities and basis vectors
 - All zero and one vectors

 - Sum
 - Scalar Multiple
 - Magnitude or Norm of a vector
 - P-Norm of a vector
 - Manhattan, Euclidean, Absolute Value, and Infinite Norm
 - Dot (inner) product
 - Orthogonality
 - Cross product
 - Scalar triple product
- Matrices
 - Intro
 - Set of all matrices
 - Addition/Subtraction
 - Scalar multiplication
 - Matrix multiplication
 - Identity matrix and Kronecker delta
 - Inverse
 - Pseudoinverse
 - Hadamard product
 - Matrix of all 1s
 - Transpose
 - Trace
 - Determinant
 - Rank and Nullity
 - Matrix Powers
 - Matrix Exponential
 - Tensor / Kronecker Product

- Direct Sum
- Eigenvalues
- Singular Values
- Matrix Norms
- Spectral Radius
- Condition Number
- Tensors
 - Introduction
 - Einstein Notation

1.2 Notes

1. The notebooks are largely self-contained, i.e, if a symbol appears, an explanation will be present at some point in the notebook.
 - Most often there will be links to the cell where the symbols are explained
 - If the symbols are not explained in this notebook, a reference to the appropriate notebook will be provided
2. **Github does a poor job of rendering this notebook.** The online render of this notebook is missing links, symbols, and notations are badly formatted. It is advised to clone a local copy (or download the notebook) and open it locally.
3. **See the Collections notebook before this notebook to gain familiarity with set notations**

1.3 Importing Libraries

```
[40]: import random
import math
import collections

def get_rand_num_btw(n):
    return round(random.random() * random.randint(-n, n), 4)
```

Vectors

Introduction

A (real) vector is a representation of n real numbers in a single column (may be represented in a single row for notational convenience):

$$\mathbf{x} = \vec{x} = \underline{x} = x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_n \end{bmatrix}$$

Vectors are often denoted using bold lower case letters such as $\mathbf{v}$, but can also be denoted using an arrow on top of a normal case letter such as $\vec{v}$. Some authors may also denote them with a tilde underline such as $\underline{v}$, and some mathematicians may simply use a lower case normal case letter, v .

```
[41]: #x is a vector of length 6

x = [2 ,3, 4, 10, -3, 5]

print('Length of vector: ', len(x))
x
```

Length of vector: 6

```
[41]: [2, 3, 4, 10, -3, 5]
```

Set of all vectors

The set of all real vectors of length n real numbers is denoted

$$\mathbb{R}^n$$

The set of all complex vectors of length n complex numbers is denoted

$$\mathbb{C}^n$$

Please see Numbers notebook for a higher dimensional numbers set perspective of this concept

```
[42]: # Small subset from R^3, each 3-tuple in the set is a single vector

no_of_samples = 10
n = 3

R_n = set()
for j in range(no_of_samples):
    vec = []
    for i in range(n):
        vec.append(get_rand_num_btw(10))
    R_n.add(tuple(vec))

R_n
```

```
[42]: {(-1.0751, -0.5965, -5.1846),
      (-0.7171, 0.2381, 0.0392),
      (-0.4447, -5.6364, -6.0413),
      (0.0, -0.6283, -0.767),
      (0.1495, -2.4781, 0.5995),
      (0.422, 2.7613, 8.1899),
```

(0.4564, -0.042, -4.6766),
 (0.6893, 1.4437, 0.7642),
 (1.9371, 5.2027, -0.2391),
 (5.5206, 0.7351, -0.9093)}

Physical quantities and basis vectors

Vectors that are used to represent physical quantities such as displacement, velocities, forces etc are typically represented in three dimensions due to the use of Euclidean space and hence are elements of $\mathbb{R}^3$ (See Set of all vectors).

Such vectors are often expressed using basis vectors or $\mathbf{i} - \mathbf{j} - \mathbf{k}$ coordinates. These basis vectors are defined as:

$$\mathbf{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \mathbf{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \mathbf{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Basis vectors (or *unit vectors* in general) have magnitude 1 (See Magnitude or Norm of a vector) and can also be denoted using hats on top of the letters $\hat{\mathbf{i}}$ such that:

$$\hat{\mathbf{i}} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \hat{\mathbf{j}} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \hat{\mathbf{k}} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

The basis vectors can then be used to represent a three dimensional vector $\mathbf{v} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$ as:

$$\mathbf{v} = a\hat{\mathbf{i}} + b\hat{\mathbf{j}} + c\hat{\mathbf{k}}$$

```
[43]: v = [2, 3, 4]

i = [1,0,0]
j = [0,1,0]
k = [0,0,1]

A = [i,
      j,
      k]

new_v = [0,0,0] #Initializing

for val in range(len(new_v)):
    new_v[val] = sum([ a*v[idx] for idx, a in enumerate(A[val])])

v, new_v
```

[43]: ([2, 3, 4], [2, 3, 4])

In $\mathbb{R}^n$ the standard basis vectors are denoted using: $\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_n$ where $\mathbf{e}_j$ is a vector with all 0s except for a single 1 at position j such that the norm of each basis vector is 1 and are called *unit vectors*. (See Magnitude or Norm of a vector)

$$\hat{\mathbf{e}}_1 = \begin{bmatrix} 1 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \hat{\mathbf{e}}_2 = \begin{bmatrix} 0 \\ 1 \\ \vdots \\ 0 \\ 0 \end{bmatrix}, \dots, \hat{\mathbf{e}}_{n-1} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix}, \hat{\mathbf{e}}_n = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}$$

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_{n-1} \\ v_n \end{bmatrix}, \mathbf{v} = v_1 \hat{\mathbf{e}}_1 + v_2 \hat{\mathbf{e}}_2 + \dots + v_{n-1} \hat{\mathbf{e}}_{n-1} + v_n \hat{\mathbf{e}}_n$$

All zero and one vectors

A vector whose elements are all 0s are denoted using: $\mathbf{0}$, $\vec{0}$, or simply 0, although 0 is discouraged since it is difficult to distinguish it from the number 0.

$$\mathbf{0} = \vec{0} = 0 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{bmatrix}$$

A vector whose elements are all 1s are denoted using: $\mathbf{1}$, $\vec{1}$, $\mathbf{e}$ or $\vec{e}$.

$$\mathbf{1} = \vec{1} = \mathbf{e} = \vec{e} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

[44]: *#All zero and all one vectors of length 6*

```
all_zero = [0,0,0,0,0,0]
all_one = [1,1,1,1,1,1]

all_zero, all_one
```

[44]: ([0, 0, 0, 0, 0, 0], [1, 1, 1, 1, 1, 1])

Vector sum

Two (or more) vectors, say $\mathbf{x}$ and $\mathbf{y}$, of the same length, say n numbers, can be summed and it is denoted:

$$\mathbf{x} + \mathbf{y}$$

```
[45]: x = [2 , 3.5, 4, 10.6, -3, 5.5]
      y = [1 , 8, 2, -10, 0, 2.5]

      sum_vec = [a + b for a,b in zip(x,y)] #x + y
      sum_vec
```

```
[45]: [3, 11.5, 6, 0.5999999999999996, -3, 8.0]
```

Scalar multiple

The scalar multiple where a scalar s is multiplied to a vector, $\mathbf{x}$ is denoted:

$$s\mathbf{x}$$

```
[46]: x = [2 , 3.5, 4, 10.6, -3, 5.5]
      s = 4

      scal_mul = [s*a for a in x] #sx
      scal_mul
```

```
[46]: [8, 14.0, 16, 42.4, -12, 22.0]
```

Magnitude or Norm of a vector

The magnitude or norm of a vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$ is denoted as:

$$\|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_{n-1}^2 + x_n^2}$$

```
[47]: x = [0, 4, 3]

      norm = math.sqrt(sum([a**2 for a in x]))

      norm
```

[47]: 5.0

Note 1: This may also be called length of the vector by some authors in which case they are referring to the norm and not to the number of elements in the vector

Note 2: Some authors use single bars instead of double bars to denote the norm. This may cause confusion when referring to absolute value which is part of the definition of norms in general (See P-Norm of a vector).

$$|\mathbf{x}| = \sqrt{x_1^2 + x_2^2 + \dots + x_{n-1}^2 + x_n^2}$$

Note 3: In general, p -norms exist, but the 2-norm is typically denoted without the 2 explicitly, whereas the 1-norm is denoted with the 1 explicitly (See P-Norm of a vector)

$$\|\mathbf{x}\|_2 = \|\mathbf{x}\|$$

$$\|\mathbf{x}\|_1 \neq \|\mathbf{x}\|$$

Note 4: For complex vectors it is better to look at the general definition of the norm, which is the p -norm (See P-Norm of a vector)

P-Norm of a vector

This is the generalization of the concept of the magnitude or norm (See Magnitude or Norm of a vector).

For a positive real number p , the p -norm of vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$ is:

$$\|x\|_p = [|x_1|^p + |x_2|^p + \dots + |x_{n-1}|^p + |x_n|^p]^{1/p}$$

where $|x_j|$ denotes absolute value

```
[48]: x = [0, 4, 3]
      p = 2

      pnorm = math.pow(sum([abs(a)**p for a in x]), 1/p)

      'p: ', p, 'x: ', x, 'pnorm: ', pnorm
```

```
[48]: ('p: ', 2, 'x: ', [0, 4, 3], 'pnorm: ', 5.0)
```

Manhattan, Euclidean, Absolute Value, and Infinity Norm

From the generalization of the p -norm, some specific values of p can have many applications in various fields and so certain values of p -norm have special names. (See P-Norm of a vector)

For the explanations assume a vector $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$

When $p = 1$, this norm is called the Manhattan norm or the Taxicab norm or the ℓ^1 norm (pronounced ell 1).

$$\|\mathbf{x}\|_1 = |x_1| + |x_1| + \dots + |x_{n-1}| + |x_n| = \sum_{i=1}^n |x_i|$$

When $p = 2$, this is called the Euclidean norm, or magnitude of a vector or the ℓ^2 norm or simply the norm of the vector (See Magnitude or Norm of a vector). This is generally denoted without the subscript 2.

$$\|\mathbf{x}\|_2 = \|\mathbf{x}\| = \sqrt{x_1^2 + x_2^2 + \dots + x_{n-1}^2 + x_n^2} = \sqrt{\sum_{i=1}^n x_i^2}$$

When $p = \infty$, this is called the Infinity norm or Maximum norm and is defined as:

$$\|\mathbf{x}\|_\infty = \max \{|x_1|, |x_1|, \dots, |x_{n-1}|, |x_n|\}$$

If the vector is one dimensional (a scalar), the p -norm of the vector is the absolute value. In the example below, the one dimensional vector is not denoted using bold face indicating it is a scalar.

$$\|x\| = |x|$$

```
[49]: terms = {
      1: 'Manhattan',
      2: 'Euclidean',
      'inf': 'Infinity'
    }

def norms(p, x, terms):
    if p == 'inf':
        norm = max([abs(a) for a in x]) #infinity norm
    else:
        norm = round( math.pow(sum([abs(a)**p for a in x]), 1/p) , 2) #general
        ↪p-norm calc
```



```

if len(x) == 1:
    name = 'Absolute value' #one dimensional vector case
elif p in terms.keys():
    name = terms[p]
else:
    name = str(p)
print(f'The {name} norm is {norm}\n')

x = [0,3,4]
p = 1
print('x: ', x)
print('p is', p)
norms(p, x, terms)

p = 2
print('x: ', x)
print('p is', p)
norms(p, x, terms)

p = 3
print('x: ', x)
print('p is', p)
norms(p, x, terms)

p = 'inf'
print('x: ', x)
print('p is', p)
norms(p, x, terms)

x = [-4]
p = 3
print('x: ', x)
print('p is', p)
norms(p, x, terms)

```

```

x:  [0, 3, 4]
p is 1
The Manhattan norm is 7.0

```

```

x:  [0, 3, 4]
p is 2
The Euclidean norm is 5.0

```

```

x:  [0, 3, 4]
p is 3
The 3 norm is 4.5

```

```

x:  [0, 3, 4]

```

```
p is inf
The Infinity norm is 4
```

```
x: [-4]
p is 3
The Absolute value norm is 4.0
```

Dot (inner) product

The inner product or dot product of two vectors $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$ and $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \\ y_n \end{bmatrix}$ is the scalar

$$\mathbf{x} \cdot \mathbf{y} = x_1 y_1 + x_2 y_2 + \dots + x_{n-1} y_{n-1} + x_n y_n = \sum_{k=1}^n x_k y_k$$

This should not be confused with an elementwise product, which multiplies matching coordinates without summing them.

Sometimes, the bullet glyph is used to represent dot products $\mathbf{x} \bullet \mathbf{y}$ and the smaller dot is used to represent multiplication in general.

$$\mathbf{x} \bullet \mathbf{y} = x_1 \cdot y_1 + x_2 \cdot y_2 + \dots + x_{n-1} \cdot y_{n-1} + x_n \cdot y_n = \sum_{k=1}^n x_k \cdot y_k$$

The dot product may also be denoted using angle brackets $\langle \mathbf{x}, \mathbf{y} \rangle$, or using the transpose notations $\mathbf{x}^t \mathbf{y}$ or $\mathbf{x}^T \mathbf{y}$ (See Transpose).

If $\mathbf{x}$ and $\mathbf{y}$ are complex n -vectors, then often the inner product is represented as:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{k=1}^n x_k \overline{y_k}$$

although some authors define it as:

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{k=1}^n \overline{x_k} y_k$$

where $\overline{x}$ denotes complex conjugate.

See the Numbers notebook to learn more about complex conjugation.

```
[50]: x = [2,3,6]
      y = [5,2,1]
```

```
def dot_product(x,y):
    return sum([a*b for a,b in zip(x,y)])

x_dot_y = dot_product(x,y)
x_dot_y
```

[50]: 22

Note 1: Some literature from the physics community also denote inner product as:

$$\langle \mathbf{x} | \mathbf{y} \rangle$$

This is known as the *bra-ket* notation where the $\langle x |$ is the row vector called the bra-vector and the $| y \rangle$ is the column vector called the ket-vector.

Note 2: The dot product of two vectors $\mathbf{x}$ and $\mathbf{y}$ (mostly in $\mathbb{R}^2$ or $\mathbb{R}^3$) with angle θ between them is calculated using the formula:

$$\mathbf{x} \cdot \mathbf{y} = \|\mathbf{x}\| \|\mathbf{y}\| \cos \theta$$

See Magnitude or Norm of a vector

Note 3: Inner products arise in a context beyond $\mathbb{R}^n$ and $\mathbb{C}^n$ such as in calculus where $f, g : \mathbb{R} \rightarrow \mathbb{R}$ then

$$\langle f, g \rangle = \int_{-\infty}^{\infty} f(t)g(t) dt$$

For function notations f please see the [Functions notebook](#)

For more information on the calculus perspective please see the [Calculus notebook](#)

Orthogonality

Two vectors are called orthogonal if their dot product is zero, which then can be denoted as:

$$\mathbf{x} \perp \mathbf{y}$$

```
[51]: x = [ 2, -1, 0]
      y = [ 1, 2, 0]

def is_orthogonal(x,y):
    return round(dot_product(x,y), 4) == 0

print('x ',x)
```

```

print('y ',y)
print('Is orthogonal: ',is_orthogonal(x,y))

x = [ 5, 2, 0]
y = [-3, -10, 0]

print('x ',x)
print('y ',y)
print('Is orthogonal: ',is_orthogonal(x,y))

```

```

x [2, -1, 0]
y [1, 2, 0]
Is orthogonal: True
x [5, 2, 0]
y [-3, -10, 0]
Is orthogonal: False

```

The $\perp$ symbol is also used to define subspaces of $\mathbb{R}^n$. If $\mathcal{V}$ is a subspace of $\mathbb{R}^n$, then $\mathcal{V}^\perp$ is the set of vectors that are orthogonal to all vectors in $\mathcal{V}$:

$$\mathcal{V}^\perp = \{\mathbf{x} \in \mathbb{R}^n \mid \forall \mathbf{v} \in \mathcal{V}, \mathbf{x} \perp \mathbf{v}\}$$

Then the subspace $\mathcal{V}^\perp$ is called the orthogonal complement of $\mathcal{V}$

Please see the Logic notebook for more on the $\forall$ notation

Cross product

The cross product of two vectors, $\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ and $\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$ such that $\mathbf{x}, \mathbf{y} \in \mathbb{R}^3$ is denoted as:

$$\mathbf{x} \times \mathbf{y} = \begin{bmatrix} x_2 y_3 - x_3 y_2 \\ x_3 y_1 - x_1 y_3 \\ x_1 y_2 - x_2 y_1 \end{bmatrix}$$

The cross product is defined only for vectors in $\mathbb{R}^3$

In some applied mathematics and physics literature, the wedge glyph may be used to denote cross product:

$$\mathbf{x} \wedge \mathbf{y} = \begin{bmatrix} x_2 y_3 - x_3 y_2 \\ x_3 y_1 - x_1 y_3 \\ x_1 y_2 - x_2 y_1 \end{bmatrix}$$

```

[52]: x = [1, 2, 1]
      y = [2, 2, 2]

```

```

# x cross y in R^3
cross_product = [
    x[1] * y[2] - x[2] * y[1],
    x[2] * y[0] - x[0] * y[2],
    x[0] * y[1] - x[1] * y[0],
]

print('x =', x)
print('y =', y)
print('x cross y =', cross_product)

```

```

x = [1, 2, 1]
y = [2, 2, 2]
x cross y = [2, 0, -2]

```

Note 1: The cross product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^3$ with angle θ between them is calculated using the formula:

$$\mathbf{x} \times \mathbf{y} = (\|\mathbf{x}\| \|\mathbf{y}\| \sin \theta) \hat{\mathbf{n}}$$

where $\hat{\mathbf{n}}$ is the unit vector that is perpendicular to the plane of $\mathbf{x}$ and $\mathbf{y}$
(See Physical quantities and basis vectors for more on unit vectors)

Scalar triple product

Given three vectors $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$, their *scalar triple product* is defined as:

$$[\mathbf{a}, \mathbf{b}, \mathbf{c}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$$

It can also be computed as the determinant of the matrix whose rows (or columns) are the three vectors:

$$[\mathbf{a}, \mathbf{b}, \mathbf{c}] = \det \begin{bmatrix} a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 \\ c_1 & c_2 & c_3 \end{bmatrix}$$

The absolute value of the scalar triple product equals the volume of the parallelepiped spanned by the three vectors.

```

[53]: a = [1, 0, 0]
      b = [0, 1, 0]
      c = [0, 0, 1]

      def cross_product(x, y):
          return [x[1]*y[2] - x[2]*y[1],
                  x[2]*y[0] - x[0]*y[2],

```

```
x[0]*y[1] - x[1]*y[0]]
```

```
scalar_triple = dot_product(a, cross_product(b, c))
print(f'[a, b, c] = a . (b x c) = {scalar_triple}')
```

```
[a, b, c] = a . (b x c) = 1
```

Matrices

Introduction

A matrix is a rectangular array representation of numbers which is generally represented with m rows and n columns which would then be called a $m \times n$ matrix. General notations for matrices denote the number of rows first and the number of columns second (m rows $\times$ n columns).

If A is a $m \times n$ matrix, the entry in the i^{th} row and j^{th} column is variously denoted as $A_{i,j}$ or $a_{i,j}$ or $[A]_{i,j}$. It is very common to omit the comma separating the subscripts such as: a_{ij}

$$A_{m,n} = \begin{bmatrix} a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

```
[54]: m = 5
      n = 3

      B = [[0] * n] * m #Simple representation of matrix
      B
```

```
[54]: [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
[55]: def rand_matrix(m, n):
      M = []
      for i in range(m):
          row = []
          for j in range(n):
              row.append(get_rand_num_btw(1000))
          M.append(row)
      return M

      def show_matrix(M):
          for row in M:
              print(row)
          print()

      def transpose(M):
          rows = len(M)
          cols = len(M[0])
```

```

T = []
for j in range(cols):
    row = []
    for i in range(rows):
        row.append(M[i][j])
    T.append(row)
return T

A = rand_matrix(4, 3)
m = len(A)
n = len(A[0])
print('Matrix A, m by n: ', m, 'by', n)
show_matrix(A)
print('Element A_1_1')
print(A[0][0])
print('\nMatrix A represented as nested list')
A

```

Matrix A, m by n: 4 by 3
 [461.0481, 81.4672, 623.5099]
 [-245.6852, 346.2829, -469.1688]
 [-161.8254, 154.7715, -291.0909]
 [219.9629, -487.0801, 291.0716]

Element A_1_1
 461.0481

Matrix A represented as nested list

```
[55]: [[461.0481, 81.4672, 623.5099],
      [-245.6852, 346.2829, -469.1688],
      [-161.8254, 154.7715, -291.0909],
      [219.9629, -487.0801, 291.0716]]
```

Note 1: Vectors of n elements can be represented as a $n \times 1$ matrix (column vector) or a $1 \times n$ matrix (row vector)

```
[56]: A = rand_matrix(4, 1)
print('Matrix A, m by n: ', len(A), 'by', len(A[0]))
show_matrix(A)

B = rand_matrix(1, 4)
print('Matrix B, m by n: ', len(B), 'by', len(B[0]))
show_matrix(B)

```

Matrix A, m by n: 4 by 1
 [684.0893]
 [-165.0906]

```
[-338.1042]
[533.8851]
```

Matrix B, m by n: 1 by 4
[90.6405, 39.9262, 459.5513, -177.3663]

Note 2: A *square matrix* has the same number of rows and columns, i.e, if A is a square matrix, then shape of A is such that $m = n$

```
[57]: n = 3
      A = rand_matrix(n, n)

      print('A is a square matrix of shape: ', n, '\n')
      show_matrix(A)
```

A is a square matrix of shape: 3

```
[-593.5239, 547.9319, -559.9783]
[-266.9619, 605.1127, -121.2059]
[23.6348, -250.5505, 725.966]
```

Set of all matrices

The set of all real $m \times n$ matrices is denoted

$$\mathbb{R}^{m \times n}$$

An alternative notation for the set of all real $n \times n$ square matrices is $M_n(\mathbb{R})$ and for all real $m \times n$ matrices is $M_{m,n}(\mathbb{R})$

The set of all complex $m \times n$ matrices is denoted

$$\mathbb{C}^{m \times n}$$

Similar to the set of real matrices, an alternative notation for the set of all complex $n \times n$ square matrices is $M_n(\mathbb{C})$ and for all complex $m \times n$ matrices is $M_{m,n}(\mathbb{C})$

```
[58]: # Small subset from  $\mathbb{R}^{(3 \times 3)}$ 

no_of_samples = 10
m, n = 3, 3

R_m_n = set()
for _ in range(no_of_samples):
    M = rand_matrix(m, n)
    # convert to tuple of tuples for hashing
    M_tup = tuple(tuple(row) for row in M)
```



```
R_m_n.add(M_tup)
```

```
R_m_n
```

```
[58]: {((-655.1075, -135.1, -4.7512),
        (42.9608, 820.431, -64.7563),
        (267.149, 307.6506, 365.4986)),
       ((-188.1723, -383.7149, 290.0379),
        (61.9836, -48.9304, -48.3794),
        (173.4937, 0.4357, -31.6532)),
       ((-168.2885, 214.377, 595.6218),
        (35.0273, 622.8046, 16.4412),
        (706.861, -476.7116, -102.8595)),
       ((-29.5787, -18.2103, 135.4847),
        (243.2361, -60.6533, -67.654),
        (-19.5097, -527.3541, -187.0687)),
       ((87.3201, 501.5089, -384.3851),
        (90.8346, 189.3703, -456.5508),
        (543.3052, 596.1591, -376.6234)),
       ((110.2494, 3.6555, 571.5782),
        (33.8258, 123.1271, 354.89),
        (-910.6435, 2.9817, 206.7129)),
       ((185.3955, 7.5563, 752.662),
        (38.0876, 216.9779, -671.896),
        (-130.9708, 30.7705, 47.8957)),
       ((323.2744, -490.2396, 164.1125),
        (227.0025, 397.3555, 9.3047),
        (358.7699, 406.4405, 237.0946)),
       ((438.4064, 199.4145, -470.4426),
        (14.8712, 26.023, -119.7639),
        (242.8667, 104.4802, -42.0366)),
       ((511.3988, -24.9216, 1.9138),
        (-158.0434, -35.2496, 69.6572),
        (454.5386, -315.028, -616.2028))}
```

Addition/Subtraction

Matrices of the same shape may be added or subtracted wherein the i, j entry of each of the matrices are individually added or subtracted such that $C = A + B$ where A, B are $m \times n$ matrices, is effectively achieved by

$$c_{i,j} = a_{i,j} + b_{i,j}$$

```
[59]: def mat_add(A, B):
        assert len(A) == len(B) and len(A[0]) == len(B[0]), 'Matrix shapes do not
        ↪match'
        C = []
```

```

    for i in range(len(A)):
        row = []
        for j in range(len(A[0])):
            row.append(A[i][j] + B[i][j])
        C.append(row)
    return C

A = [[1, 2, 3],
      [3, 4, 5]]
print('Matrix A')
show_matrix(A)

B = [[6, 7, 8],
      [8, 9, 10]]
print('Matrix B')
show_matrix(B)

C = mat_add(A, B)
print('Matrix C')
show_matrix(C)

```

Matrix A

[1, 2, 3]

[3, 4, 5]

Matrix B

[6, 7, 8]

[8, 9, 10]

Matrix C

[7, 9, 11]

[11, 13, 15]

Scalar multiplication

If A is a $m \times n$ matrix and r is a scalar, then $B = rA$ is a matrix of the same shape whose i, j entry can each individually be calculated as:

$$b_{i,j} = ra_{i,j}$$

```

[60]: def scal_mul(r, A):
        B = []
        for row in A:
            new_row = []
            for a_ij in row:

```

```

        new_row.append(r * a_ij)
    B.append(new_row)
    return B

A = [[1, 2, 3],
      [3, 4, 5]]
print('Matrix A')
show_matrix(A)

r = 10
print('r is: ', r)

print('\nMatrix B')
B = scal_mul(r, A)
show_matrix(B)

```

```

Matrix A
[1, 2, 3]
[3, 4, 5]

```

```

r is:  10

```

```

Matrix B
[10, 20, 30]
[30, 40, 50]

```

Matrix multiplication

If A is a $m \times n$ matrix and B is a $n \times p$ matrix, then they may be multiplied such that $C = AB$ where C will be a $m \times p$ matrix wherein the new matrix elements will be calculated as:

$$c_{i,j} = \sum_{k=1}^n a_{i,k} b_{k,j}$$

```

[61]: def mat_mul(A, B):
        assert len(A[0]) == len(B), 'Matrix shapes are inconsistent and cannot be
        multiplied'
        B_T = transpose(B)
        C = []
        for a_i in A:
            row = []
            for b_j in B_T:
                # dot product of row a_i and column b_j
                val = 0
                for a_ik, b_kj in zip(a_i, b_j):

```

```

        val += a_ik * b_kj
    row.append(val)
    C.append(row)
return C

A = [[1, 2, 3],
     [4, 5, 6]]

B = [[1, 2],
     [3, 4],
     [5, 6]]

print('Matrix A')
show_matrix(A)
print('Matrix B')
show_matrix(B)
print('Matrix C = AB')
C = mat_mul(A, B)
show_matrix(C)

```

```

Matrix A
[1, 2, 3]
[4, 5, 6]

```

```

Matrix B
[1, 2]
[3, 4]
[5, 6]

```

```

Matrix C = AB
[22, 28]
[49, 64]

```

Identity matrix and Kronecker delta

The Identity matrix is an $n \times n$ matrix whose diagonal elements are 1 and 0 otherwise and is denoted using just I when the dimensions are understood by context, or I_n when the number of dimensions must be explicitly specified:

$$I = I_n = \begin{bmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{bmatrix}$$

The Kronecker's delta $\delta_{i,j}$ is used to represent the i, j element of an identity matrix. It can be very useful to represent the identity matrix element in general formulas since the value of the Kronecker

delta is 1 when $i = j$ and 0 otherwise. It can also be denoted as $\delta(i, j)$.

$$\delta_{i,j} = \delta(i, j) = \begin{cases} 1, & \text{if } i = j \\ 0, & \text{if } i \neq j \end{cases}$$

See the functions notebook for more on the piecewise function notation

```
[62]: def get_identity_matrix(n):  
    I = []  
    for i in range(n):  
        row = []  
        for j in range(n):  
            row.append(1.0 if i == j else 0.0)  
        I.append(row)  
    return I
```

```
I = get_identity_matrix(3)  
show_matrix(I)
```

```
[1.0, 0.0, 0.0]  
[0.0, 1.0, 0.0]  
[0.0, 0.0, 1.0]
```

```
[63]: def kronecker_delta(i, j):  
    assert all([i > 0, j > 0]), 'Values of i and/or j are inappropriate'  
    return 1.0 if i == j else 0.0
```

```
kronecker_delta(2, 2)
```

```
[63]: 1.0
```

Inverse

If A and B are square matrices such that $AB = I$, then B is called the inverse of A and is denoted as A^{-1} .

$$AA^{-1} = A^{-1}A = I$$

where A^{-1} is the inverse of A

(See Identity matrix and Kronecker delta for more info on I)

```
[64]: def round_Mvalues(A, digits):  
    result = []  
    for row in A:  
        new_row = []
```

```

        for val in row:
            new_row.append(round(val, digits))
        result.append(new_row)
    return result

def is_inverse(A, B):
    C = mat_mul(A, B)
    roundedC = round_Mvalues(C, 2)
    I = get_identity_matrix(len(C))
    return I == roundedC

A = [[4, 7],
      [2, 6]]

B = [[.6, -.7],
      [-.2, .4]]

print('Matrix A')
show_matrix(A)

print('Matrix B')
show_matrix(B)

print('Is B the inverse of A: ', is_inverse(A, B))

```

Matrix A

[4, 7]

[2, 6]

Matrix B

[0.6, -0.7]

[-0.2, 0.4]

Is B the inverse of A: True

Pseudoinverse

The Moore-Penrose pseudoinverse of a Matrix A is denoted as

$$A^+$$

Note 1: Although some authors also use $A^\dagger$ for the pseudoinverse, this symbol is more often used to denote the conjugate transpose of A . (See Transpose in this notebook)

Hadamard product

If A and B are two matrices of shape $m \times n$, then the Hadamard product of A and B is a matrix C also of shape $m \times n$ consisting of the elementwise product such that $c_{ij} = a_{ij}b_{ij}$. It is denoted as:

$$C = A \circ B$$

This is also called the elementwise product of matrices.

```
[65]: def hadamard_product(A, B):
        assert len(A) == len(B) and len(A[0]) == len(B[0]), 'Matrix shapes are
        inconsistent'
        C = []
        for i in range(len(A)):
            row = []
            for j in range(len(A[0])):
                row.append(A[i][j] * B[i][j])
            C.append(row)
        return C

A = [[10, 20, 30],
      [15, 30, 45]]

B = [[2, 4, 6],
      [2, 4, 5]]

print('Matrix A')
show_matrix(A)

print('Matrix B')
show_matrix(B)

C = hadamard_product(A, B)

print('The hadamard product of A and B is:')
show_matrix(C)
```

```
Matrix A
[10, 20, 30]
[15, 30, 45]
```

```
Matrix B
[2, 4, 6]
[2, 4, 5]
```

```
The hadamard product of A and B is:
[20, 80, 180]
[30, 120, 225]
```

Matrix of all 1s

A matrix of all 1s is often denoted by J . $J_{m,n}$ denotes a matrix of shape $m \times n$ whose elements are all 1's.

$$J = J_{m,n} = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix}$$

```
[66]: def matrix_all_ones(m, n):  
    J = []  
    for i in range(m):  
        row = [1.0 for j in range(n)]  
        J.append(row)  
    return J
```

```
m = 5  
n = 3  
J = matrix_all_ones(m, n)  
show_matrix(J)
```

```
[1.0, 1.0, 1.0]  
[1.0, 1.0, 1.0]  
[1.0, 1.0, 1.0]  
[1.0, 1.0, 1.0]  
[1.0, 1.0, 1.0]
```

Transpose

If A is an $m \times n$ matrix, its *transpose* is an $n \times m$ matrix denoted A' or A^T . It is defined by:

$$[A^T]_{i,j} = a_{j,i}$$

A matrix A that satisfies $A = A^T$ is called *symmetric*.

For a (complex) $m \times n$ matrix A , its *conjugate transpose* is an $n \times m$ matrix denoted A^* or A^H , defined by:

$$[A^*]_{i,j} = \overline{a_{j,i}}$$

A matrix A that satisfies $A = A^*$ is called *Hermitian* or *self-adjoint*.


```
[67]: A = [[1, 2, 3],
           [4, 5, 6]]

print('Matrix A')
show_matrix(A)
print('Transpose A^T')
A_T = transpose(A)
show_matrix(A_T)
```

```
Matrix A
[1, 2, 3]
[4, 5, 6]
```

```
Transpose A^T
[1, 4]
[2, 5]
[3, 6]
```

Trace

The *trace* of a square matrix A is the sum of its diagonal elements. If A is an $n \times n$ matrix:

$$\text{tr } A = \sum_{i=1}^n a_{i,i}$$

```
[68]: def trace(A):
        total = 0
        for i in range(min(len(A), len(A[0]))):
            total += A[i][i]
        return total

A = [[1, 2, 3],
      [4, 5, 6],
      [7, 8, 9]]

print('Matrix A')
show_matrix(A)
print('tr(A) =', trace(A))
```

```
Matrix A
[1, 2, 3]
[4, 5, 6]
[7, 8, 9]
```

```
tr(A) = 15
```

Determinant

The *determinant* of a square matrix A is denoted $\det A$. It is sometimes indicated by placing vertical bars around the matrix entries or around the matrix name: $|A|$.

For a 2×2 matrix:

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

The determinant of an $n \times n$ matrix A can be expressed by cofactor expansion along the first row:

$$\det A = \sum_{j=1}^n (-1)^{1+j} a_{1,j} M_{1,j}$$

where $M_{1,j}$ is the determinant of the $(n-1) \times (n-1)$ submatrix obtained by deleting row 1 and column j .

```
[69]: def det(mat):
      n = len(mat)
      if n == 1:
          return mat[0][0]
      if n == 2:
          return mat[0][0] * mat[1][1] - mat[0][1] * mat[1][0]
      result = 0
      for j in range(n):
          sub = [row[:j] + row[j+1:] for row in mat[1:]]
          result += ((-1) ** j) * mat[0][j] * det(sub)
      return result

A = [[1, 2, 3],
     [4, 5, 6],
     [7, 8, 0]]

print('det(A) =', det(A))
```

det(A) = 27

Rank and Nullity

The *rank* of a matrix A is the dimension of its column (or row) space and is denoted $\text{rank}(A)$.

The *null space* of A is the set of vectors $\{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$, sometimes denoted $\text{null}(A)$ or $\text{ker}(A)$. The dimension of the null space is called the *nullity*.

For an $m \times n$ matrix A , the *rank-nullity theorem* states:

$$\text{rank}(A) + \text{nullity}(A) = n$$

For more on kernels, see the **Functions notebook**.

Matrix Powers

Let A be a square matrix. For a positive integer n , A^n denotes the matrix multiplied by itself n times:

$$A^n = \underbrace{A \cdot A \cdots A}_{n \text{ times}}$$

By convention, $A^0 = I$ (the identity matrix). For negative n and invertible A :

$$A^{-n} = (A^{-1})^n$$

Given a square matrix A , the *square root* of A is a matrix B such that $B^2 = A$, written $\sqrt{A} = A^{1/2}$.

```
[70]: def mat_pow(A_vals, n):
    size = len(A_vals)
    # start with identity
    result = []
    for i in range(size):
        row = []
        for j in range(size):
            row.append(1 if i == j else 0)
        result.append(row)
    for _ in range(n):
        new = []
        for i in range(size):
            row = []
            for j in range(size):
                val = 0
                for k in range(size):
                    val += result[i][k] * A_vals[k][j]
                row.append(val)
            new.append(row)
        result = new
    return result

A = [[1, 1],
      [0, 1]]

for n in range(4):
    print(f'A^{n} = {mat_pow(A, n)}')
```

```
A^0 = [[1, 0], [0, 1]]
A^1 = [[1, 1], [0, 1]]
A^2 = [[1, 2], [0, 1]]
A^3 = [[1, 3], [0, 1]]
```

(See: Matrix Powers)

Matrix Exponential

Let A be a square matrix. The *matrix exponential* $\exp A$ (or e^A) is defined by the power series:

$$\exp A = \sum_{k=0}^{\infty} \frac{1}{k!} A^k = I + A + \frac{A^2}{2!} + \frac{A^3}{3!} + \dots$$

```
[71]: def mat_exp(A_vals, terms=10):
    size = len(A_vals)
    result = [[0.0] * size for _ in range(size)]
    # start power as identity
    power = []
    for i in range(size):
        row = []
        for j in range(size):
            row.append(1.0 if i == j else 0.0)
        power.append(row)
    for k in range(terms):
        coeff = 1 / math.factorial(k)
        for i in range(size):
            for j in range(size):
                result[i][j] += coeff * power[i][j]
        # multiply power by A
        new = []
        for i in range(size):
            row = []
            for j in range(size):
                val = 0
                for m in range(size):
                    val += power[i][m] * A_vals[m][j]
                row.append(val)
            new.append(row)
        power = new
    return result

# exp of zero matrix = identity
A = [[0, 0], [0, 0]]
print('exp(0) =', mat_exp(A, 10))

# exp of [[0,1],[0,0]]
```

```
A = [[0, 1], [0, 0]]
print('exp([[0,1],[0,0]]) =', mat_exp(A, 10))
```

```
exp(0) = [[1.0, 0.0], [0.0, 1.0]]
exp([[0,1],[0,0]]) = [[1.0, 1.0], [0.0, 1.0]]
```

Tensor / Kronecker Product

Let A be an $m \times n$ matrix and B be a $p \times q$ matrix. Their *tensor* (or *Kronecker*) product is an $mp \times nq$ matrix:

$$A \otimes B = \begin{bmatrix} a_{1,1}B & a_{1,2}B & \cdots & a_{1,n}B \\ a_{2,1}B & a_{2,2}B & \cdots & a_{2,n}B \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1}B & a_{m,2}B & \cdots & a_{m,n}B \end{bmatrix}$$

```
[72]: def kronecker_product(A, B):
    m, n = len(A), len(A[0])
    p, q = len(B), len(B[0])
    result = []
    for i in range(m):
        for s in range(p):
            row = []
            for j in range(n):
                for t in range(q):
                    row.append(A[i][j] * B[s][t])
            result.append(row)
    return result

A = [[1, 2], [3, 4]]
B = [[0, 5], [6, 7]]

R = kronecker_product(A, B)
for row in R:
    print(row)
```

```
[0, 5, 0, 10]
[6, 7, 12, 14]
[0, 15, 0, 20]
[18, 21, 24, 28]
```

Direct Sum

For any two matrices A and B , their *direct sum* is denoted $A \oplus B$. It is the block diagonal matrix:

$$A \oplus B = \begin{bmatrix} A & 0 \\ 0 & B \end{bmatrix}$$

where the 0's represent blocks of zeros of the appropriate size.

Note: The notation $A \oplus B$ is also used for the Kronecker sum $A \otimes I_b + I_a \otimes B$, where I_a and I_b are identity matrices. Context determines which meaning is intended.

```
[73]: def direct_sum(A, B):
    ma, na = len(A), len(A[0])
    mb, nb = len(B), len(B[0])
    result = []
    for i in range(ma):
        result.append(A[i] + [0] * nb)
    for i in range(mb):
        result.append([0] * na + B[i])
    return result
```

```
A = [[1, 2], [3, 4]]
B = [[5, 6, 7]]
```

```
R = direct_sum(A, B)
for row in R:
    print(row)
```

```
[1, 2, 0, 0, 0]
[3, 4, 0, 0, 0]
[0, 0, 5, 6, 7]
```

Eigenvalues

It is strongly traditional to use the letter λ for an eigenvalue of a matrix and subscripted λ 's ($\lambda_1, \lambda_2, \dots, \lambda_n$) to denote the full spectrum.

For a square matrix A , the eigenvalues are the roots of the *characteristic polynomial*:

$$\det(A - \lambda I) = 0$$

If the eigenvalues are all real, they are often subscripted in order: $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$, or the reverse. The notations $\lambda_{\min}$ and $\lambda_{\max}$ refer to the smallest and largest eigenvalues respectively.

```
[74]: # eigenvalues of a 2x2 matrix via the characteristic polynomial
# det(A - lI) = l^2 - tr(A)*l + det(A) = 0
def eigenvalues_2x2(A):
    tr = A[0][0] + A[1][1]
    d = A[0][0] * A[1][1] - A[0][1] * A[1][0]
    disc = tr ** 2 - 4 * d
    if disc >= 0:
```

```

    l1 = (tr + math.sqrt(disc)) / 2
    l2 = (tr - math.sqrt(disc)) / 2
else:
    l1 = complex(tr / 2, math.sqrt(-disc) / 2)
    l2 = complex(tr / 2, -math.sqrt(-disc) / 2)
return l1, l2

A = [[4, 2], [1, 3]]
l1, l2 = eigenvalues_2x2(A)
print(f'eigenvalues: lambda_1 = {l1}, lambda_2 = {l2}')

```

eigenvalues: lambda_1 = 5.0, lambda_2 = 2.0

(See: Eigenvalues)

(See: Transpose)

Singular Values

It is traditional to use the letters σ or s to stand for a singular value of a matrix A . The full list of singular values is usually subscripted in descending order:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_n$$

The notations $\sigma_{\max}$ and $\sigma_{\min}$ refer to the largest and smallest singular values. The singular values of A are the square roots of the eigenvalues of $A^T A$ (or $A^* A$ for complex matrices):

$$\sigma_i = \sqrt{\lambda_i(A^T A)}$$

Matrix Norms

Matrix norms use a triple vertical line to distinguish them from vector norms: $\|A\|$. However, some authors use double bars $\|A\|$.

Matrix norms satisfy the submultiplicative property:

$$\|AB\| \leq \|A\| \cdot \|B\|$$

Several specific matrix norms are commonly used:

- **Spectral norm:** $\|A\|_2 = \sigma_{\max}(A)$
- **Maximum absolute column sum norm:** $\|A\|_1 = \max_j \sum_i |a_{i,j}|$
- **Maximum absolute row sum norm:** $\|A\|_\infty = \max_i \sum_j |a_{i,j}|$
- **Frobenius norm:** $\|A\|_F = \left(\sum_{i,j} |a_{i,j}|^2\right)^{1/2}$, which is also a valid matrix norm

```
[75]: def frobenius_norm(mat):
    total = 0
    for i in range(len(mat)):
        for j in range(len(mat[0])):
            total += mat[i][j] ** 2
    return math.sqrt(total)

def col_sum_norm(mat):
    m, n = len(mat), len(mat[0])
    max_col = 0
    for j in range(n):
        col_total = 0
        for i in range(m):
            col_total += abs(mat[i][j])
        if col_total > max_col:
            max_col = col_total
    return max_col

def row_sum_norm(mat):
    max_row = 0
    for row in mat:
        row_total = 0
        for x in row:
            row_total += abs(x)
        if row_total > max_row:
            max_row = row_total
    return max_row

A = [[1, -2, 3],
     [4, 5, -6]]

print(f'||A||_F = {frobenius_norm(A):.4f}')
print(f'|||A|||_1 = {col_sum_norm(A)}')
print(f'|||A|||_inf = {row_sum_norm(A)}')
```

```
||A||_F = 9.5394
```

```
|||A|||_1 = 9
```

```
|||A|||_inf = 15
```

(See: Eigenvalues)

Spectral Radius

The *spectral radius* of a square matrix A is denoted $\rho(A)$. It is defined as:

$$\rho(A) = \max\{|\lambda| : \lambda \text{ is an eigenvalue of } A\}$$


```
[76]: A = [[4, 2], [1, 3]]
      l1, l2 = eigenvalues_2x2(A)
      rho = max(abs(l1), abs(l2))
      print(f'eigenvalues: {l1}, {l2}')
      print(f'spectral radius rho(A) = {rho}')
```

```
eigenvalues: 5.0, 2.0
spectral radius rho(A) = 5.0
```

(See: Singular Values)

Condition Number

The *condition number* of a matrix A is denoted $\kappa(A)$. It equals the ratio of the largest to smallest singular values:

$$\kappa(A) = \frac{\sigma_{\max}}{\sigma_{\min}}$$

A large condition number indicates that the matrix is nearly singular and that solving $A\mathbf{x} = \mathbf{b}$ may be numerically unstable.

```
[77]: A = [[4, 1],
          [2, 3]]

A_T = transpose(A)
A_T_A = mat_mul(A_T, A)
lambda_1, lambda_2 = eigenvalues_2x2(A_T_A)
sigma_max = math.sqrt(max(lambda_1, lambda_2))
sigma_min = math.sqrt(min(lambda_1, lambda_2))
kappa = sigma_max / sigma_min

print('A =', A)
print(f'sigma_max = {sigma_max:.4f}')
print(f'sigma_min = {sigma_min:.4f}')
print(f'kappa(A) = {kappa:.4f}')
```

```
A = [[4, 1], [2, 3]]
sigma_max = 5.1167
sigma_min = 1.9544
kappa(A) = 2.6180
```

Tensors

Introduction

In its simplest form, a *tensor* is a multidimensional array of numbers. An entry of an order-3 tensor T is denoted $T_{i,j,k}$ or T_{ijk} . Some authors use the word *rank* instead of *order*.

- Order-0 tensors are scalars
- Order-1 tensors are vectors
- Order-2 tensors are matrices
- Higher-order tensors are typically written with capital letters in italics T or boldface $\mathbf{T}$

An entry in an (n, m) -tensor is indicated with n upper indices (superscripts) and m lower indices (subscripts):

$$T_{j_1, j_2, \dots}^{i_1, i_2, \dots}$$

Kronecker's delta in tensor notation is written with one upper and one lower index:

$$\delta_j^i = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{otherwise} \end{cases}$$

```
[78]: # order-3 tensor: 2 x 3 x 4
T = [[[i + j + k for k in range(4)]
      for j in range(3)]
      for i in range(2)]

print(f'T has shape {len(T)} x {len(T[0])} x {len(T[0][0])}')
print(f'T[0][1][2] = {T[0][1][2]}')
print(f'T[1][2][3] = {T[1][2][3]}')
```

```
T has shape 2 x 3 x 4
T[0][1][2] = 3
T[1][2][3] = 6
```

Einstein Notation

Einstein notation (or the Einstein summation convention) is often used with tensors. When an index appears once as a superscript and once as a subscript, a summation over that index is implied:

$$T_k^i v^k := \sum_k T_k^i v^k$$

The repeated index is called a *dummy* index. This convention eliminates the need for explicit summation signs in many tensor equations.

For more on Einstein notation and implicit summation, see the Collections notebook.

```
[79]: # Einstein summation: matrix-vector product T_ik * v_k
T = [[1, 2], [3, 4]]
v = [5, 6]

# T^i_k v^k = sum_k T[i][k] * v[k]
result = []
```

```
for i in range(len(T)):
    total = 0
    for k in range(len(v)):
        total += T[i][k] * v[k]
    result.append(total)
print(f'T = {T}')
print(f'v = {v}')
print(f'Tik vk = {result}')
```

T = [[1, 2], [3, 4]]

v = [5, 6]

Tⁱ_k v^k = [17, 39]